altairsim — Color graphics on a Cromemco Dazzler

2026-07-25 · 849e344

Li-Chen Wang's **Kaleidoscope** (`KSCOPE`) — the classic Dazzler demo — drawing a four-way-mirrored pattern that wanders and recolors forever.

```
cd examples/dazzler
altairsim kscope.toml
```

The machine comes up **drawing**: `kscope.toml` loads KSCOPE and RUNs it from its `startup` list, on an Altair that has a **Dazzler** in it — an 8080 at 4 MHz, 64K of RAM, an 88-2SIO for the console, and the Dazzler's two ports at `0E` / `0F`. On a build with **SDL3** a window opens the moment the Dazzler turns on, and the kaleidoscope appears: a 2 KB, 64×64, 16-color picture.

KSCOPE never stops on its own (there is no `HLT`), so press **ATTN** (`Ctrl-E`) at the terminal to break back to the `altairsim>` prompt; `RUN 0` starts it again. On a **headless** build the machine runs exactly the same and simply draws nothing.

To start it by hand instead — for instance to watch it draw into memory — break out with ATTN and re-run it yourself:

```
altairsim> LOAD KSCOPE.HEX
loaded 127 bytes from KSCOPE.HEX (0000-007E)
altairsim> RUN 0
```

What KSCOPE does

The program (`KSCOPE.ASM`, with its assembler listing in `KSCOPE.PRN`) is tiny and assembles at `0000`, so it runs straight from a `RUN 0`:

- It turns the Dazzler **on** with a framebuffer at `0200` (`OUT 0EH` = `81h`), and sets the format to **2 KB, 64×64, color** (`OUT 0FH` = `30h`).
- The 2 KB picture is four 512-byte **quadrants** tiled 2×2. KSCOPE draws one pixel and then mirrors it into all four quadrants by negating each axis — which is why the pattern is symmetric about both the horizontal and vertical center. It walks a pseudo-random path and cycles the color, so the figure is always moving.

Because the framebuffer starts as whatever the RAM powered up holding (random, like real static RAM), the picture emerges from a field of color noise as KSCOPE paints over it.

The Dazzler, briefly

The Dazzler reads its picture straight out of **main memory** — the framebuffer is ordinary RAM (here at `0200`), not on the card. Two `OUT` ports drive it:

- **`OUT 0EH`** — control: bit 7 on/off, bits 6–0 the high address bits of the framebuffer (so the base is 512-byte aligned).
- **`OUT 0FH`** — format: resolution (32×32…128×128), size (512 B or 2 KB), color vs 16 greys.

`IN 0EH` reads two status bits (odd/even scan line, end-of-frame) a program can poll to pace its drawing to the frame. See `docs/boards/cromemco-dazzler.md` for the full reference.

A disk of Dazzler demos, under CP/M — and the joysticks

`kscope.toml` is the bare machine — one program, no operating system. `cpm.toml` is the game console: it boots **56K CP/M 2.2b** off an 8″ floppy with two more disks of period Dazzler software, plus a **Cromemco D+7A** so a joystick works.

```
cd examples/dazzler
altairsim cpm.toml
      -> 56K CP/M 2.2b v2.3
         A>
```

Three drives are mounted:

- **A:** `cpm22b23-56k.dsk` — the bootable CP/M system disk.
- **B:** `dazzler_graphics_altair.dsk` — `GDEMO`, `DAZZPLOT`, `GRAPHX`, `BARPLOT`, `BASIC`, and their data/source (`CUBE.DAT`, `CROMEMCO.DAT`, …).
- **C:** `dazzler_stuff_altair.dsk` — `DAZCHESS` (Dazzler chess), `KSCOPE` (the CP/M build of the kaleidoscope), `MICRO80`, and **`ADCTEST`** (the D+7A joystick diagnostic).

Change drive and run one — `B:` then `GDEMO`, or `C:` then `DAZCHESS`. On an SDL3 build the Dazzler window opens the moment a program turns the card on; `[display] focus = true` brings it to the front. It is a **display, not a keyboard** (`[display] keyboard = "none"`): what you type in the window drives the joystick, not CP/M, and only **`Ctrl-E`** in it stops the guest and hands you back the monitor — type CP/M commands in the terminal.

These are **Z80** programs (`DAZCHESS`, the graphics demos, and `ADCTEST` all use Z80 instructions), so the machine runs a **Z80** CPU — an 8080 reads those opcodes as no-ops and the programs run off into garbage.

The joysticks. The `d7a` board reads one or two Cromemco JS-1 joysticks off the host: `joystick1 = "auto"` follows the first USB gamepad you plug in, or the keyboard (arrow keys + Space/Z/X/C) if there is none. Any program that reads the D+7A ports — the buttons at `IN 18H`, the axes at `IN 19H` / `1AH` (and `1BH` / `1CH` for a second stick) — sees it. **`adctest.toml`** boots CP/M and auto-runs `ADCTEST`, which displays each stick's X/Y and buttons on the Dazzler. See `docs/boards/cromemco-d7a.md` and `reference/JS-1.md`.

The two demo disks are a collection of period Cromemco Dazzler software (graphics demos and games); `ADCTEST.COM` was copied onto the C: disk with the hostbridge file-transfer utility.

Try it yourself

- **Watch it draw into memory.** Break out (`Ctrl-E`) and `DUMP 0200` — the framebuffer bytes KSCOPE has written are right there; each byte is two 4-bit color elements.
- **Change the colors.** The picture is a palette machine: the same bytes look different under a different `OUT 0FH`. `SET daz0` shows the card; the format is set by the running program.
- **Slow it down or speed it up.** `SET cpu0 clock_hz=2000000` for a 2 MHz Altair, or `clock_hz=0` for flat out.